Course 2: Supervised Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Summary

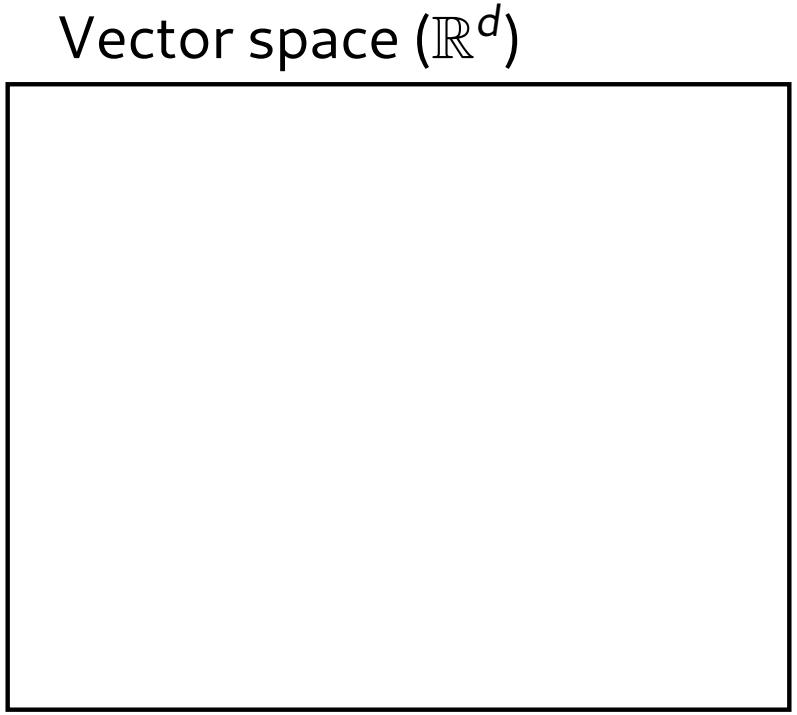
Last session

1. AI definition
2. Applications & Open Issues
3. Deep learning
4. Foundation models

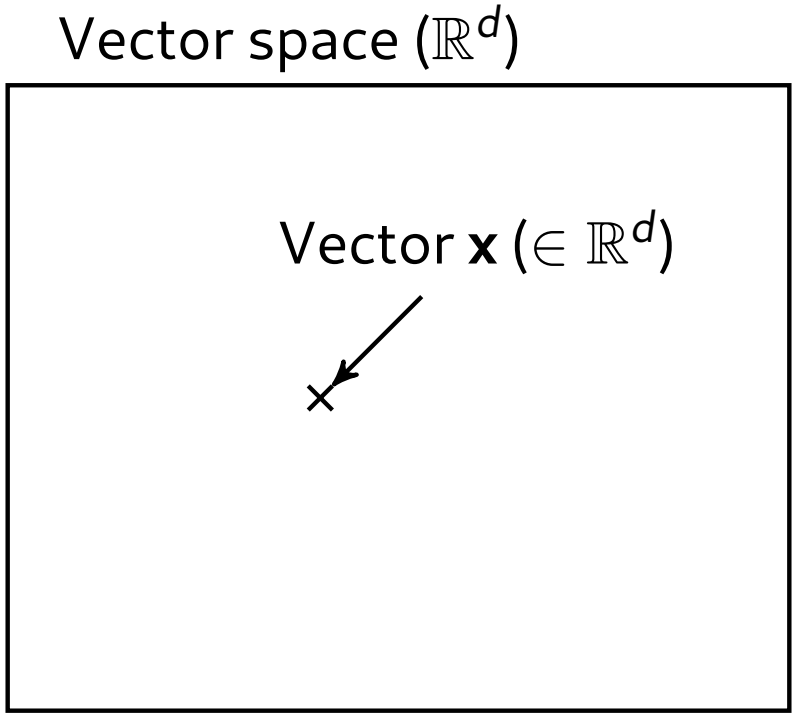
Today's session

- Learning from labeled examples
- Challenges of supervised learning
- Evaluation: ground truth and metrics

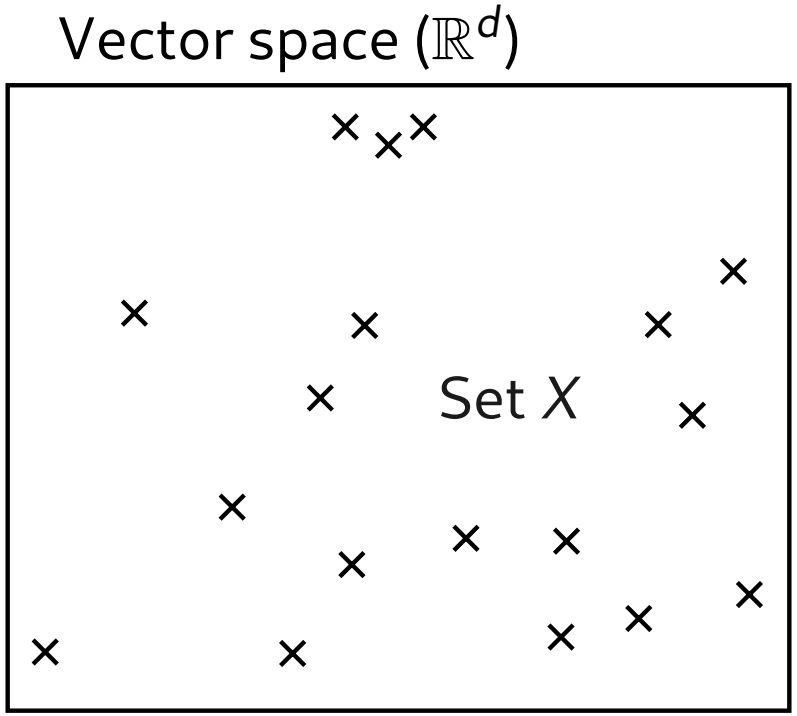
Notations



Notations



Notations



What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

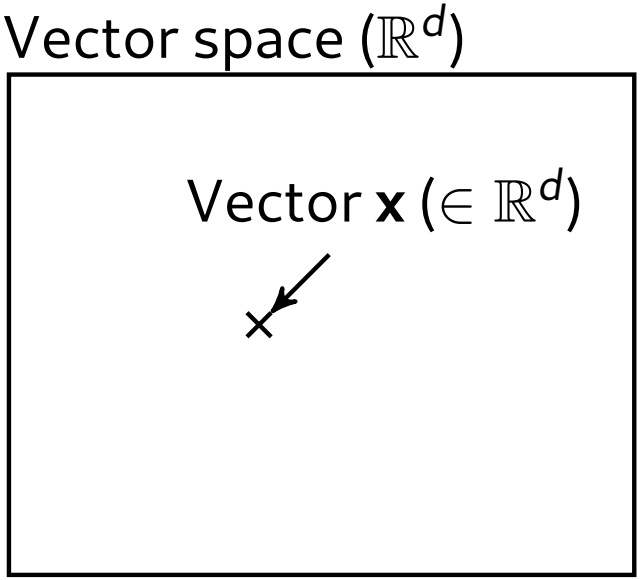
Deep Learning

$\rightarrow$ x is the data, or a small transformation of the data

The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable



What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

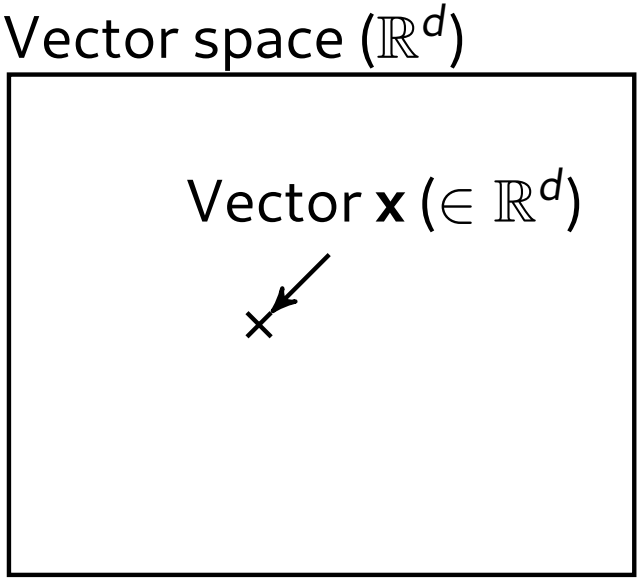
Deep Learning

$\rightarrow$ x is the data, or a small transformation of the data

The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable



What is the vector x? (1/2)

Traditional Machine Learning

$\rightarrow$ x is generally a feature representation of the data

Ex: edges in the image, spectrograms in audio, etc.

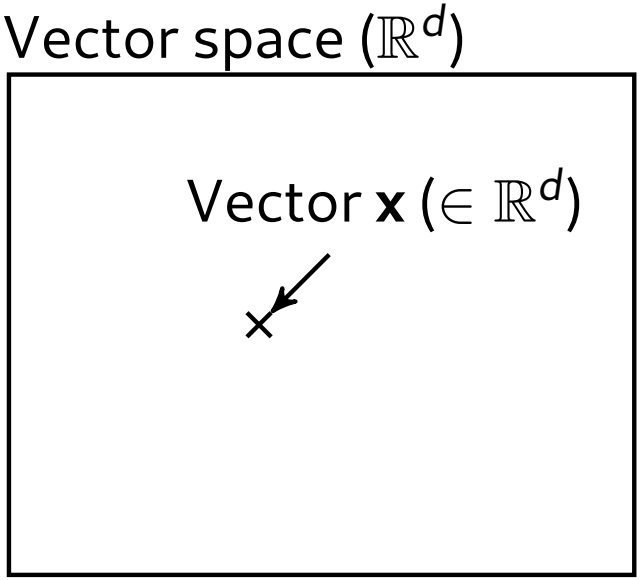
Deep Learning

$\rightarrow$ x is the data, or a small transformation of the data

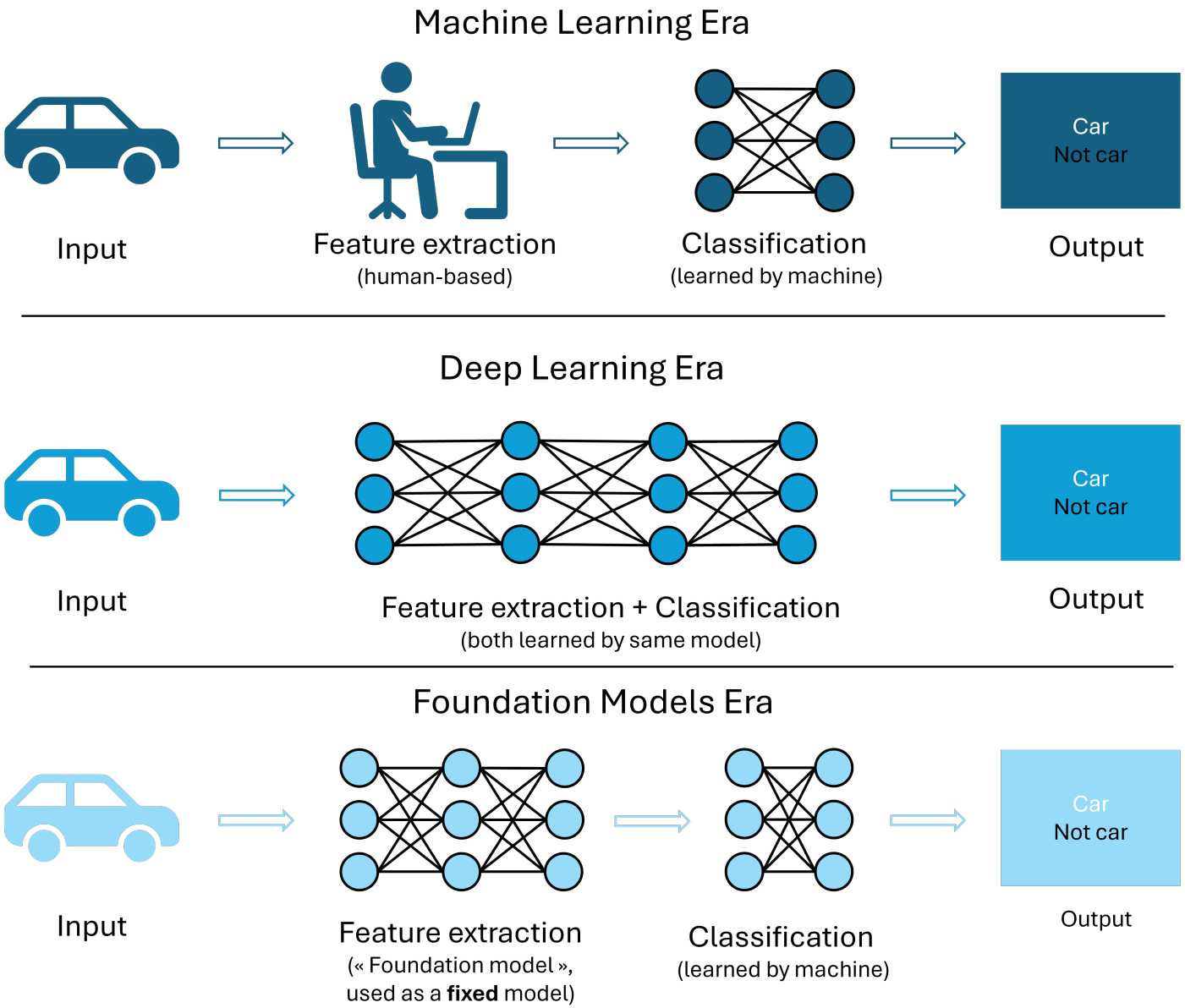
The era of Foundation models

$\rightarrow$ x is the projection of data in an **embedding** space

- Advantage: richer semantically than the original image
- Disadvantage: Not interpretable nor easily understandable



What is the vector x? (2/2)



Plan

1 Supervised learning: Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.

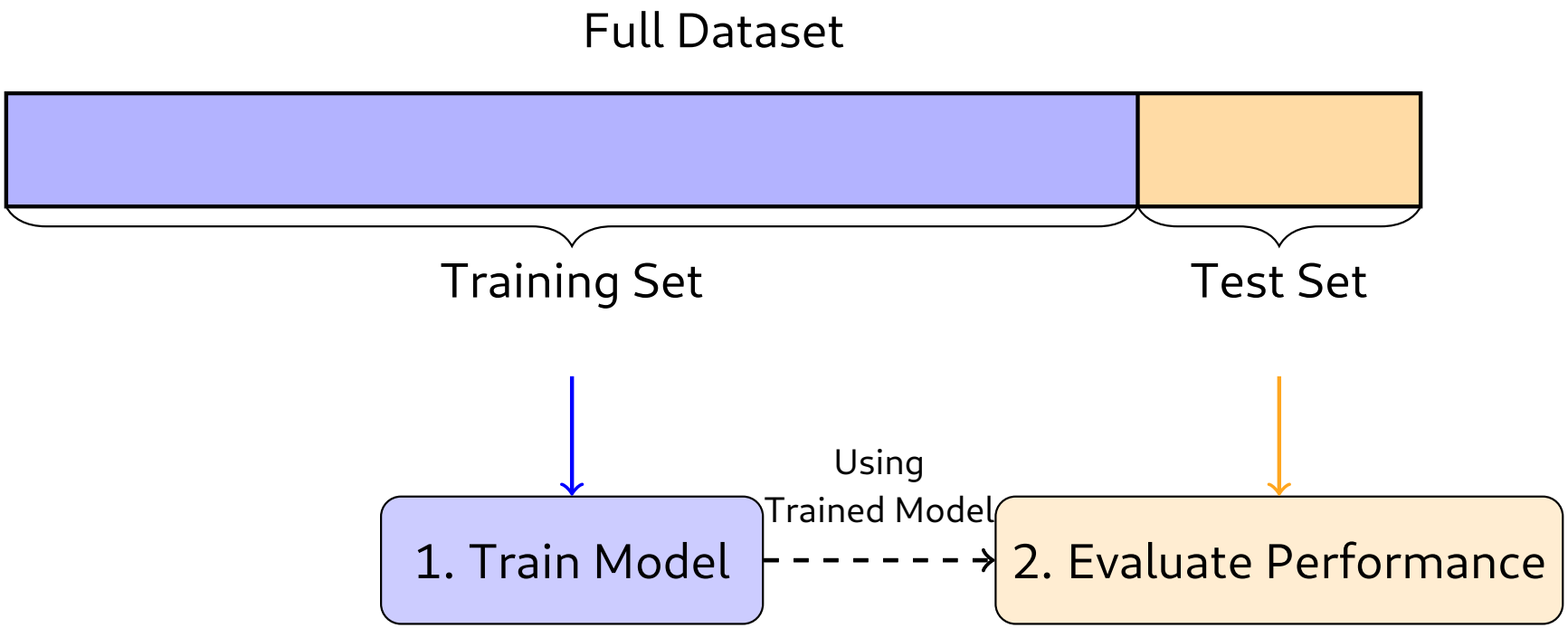
Learning: Training and Test sets

The Training Set

Data used by the model to **learn** from examples.

The Test Set

A smaller, hold-out portion that the model has **never seen**. Used to **evaluate** its performance.



Supervised learning

Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
  $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

Supervised learning

Definition

Given a training set, composed of:

- **x**: inputs (raw signals or feature vectors (e.g. embeddings))
- **y**: **labels** (annotated by humans)

Learn:

- a function $f()$ such that $\mathbf{y} \approx f(\mathbf{x})$ (denoted as $\hat{\mathbf{y}}$)
  $\Rightarrow f()$ is **learned** by the Machine Learning algorithm
- Ideally, $f()$ should **generalize** ($\neq$ memorize) to unlabeled examples.

x:  $f()$ (to be learned) → **y**: "cat"

Supervised learning: in practice

$\mathbf{x}$: 

$f()$

$\hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$

Supervised learning: in practice

$\mathbf{x}$: 

$$\xrightarrow{f()} \quad \hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Supervised learning: in practice

$\mathbf{x}$: 

$$f() \longrightarrow \hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
- Training consist in minimizing the loss!

Supervised learning: in practice

$\mathbf{x}$: 

$$\xrightarrow{f()} \quad \hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

Supervised learning: in practice

$\mathbf{x}$: 

$$f() \longrightarrow \hat{\mathbf{y}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

  $\Rightarrow$ Here, one can use gradient descent (see class 1.)

Supervised learning: in practice

$\mathbf{x}$:  $\xrightarrow{f()}$ $\hat{\mathbf{y}} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} \neq \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Loss

- Here, labels are encoded as one-hot vectors,
- We compute a **loss** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$
  - Euclidean distance: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{i=1}^{D} (\hat{\mathbf{y}}_i - \mathbf{y}_i)^2$
  - Cross-entropy: $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^{D} \mathbf{y}_i \log(\hat{\mathbf{y}}_i)$
- Training consist in minimizing the loss!

  $\Rightarrow$ Here, one can use gradient descent (see class 1.)

Supervised learning

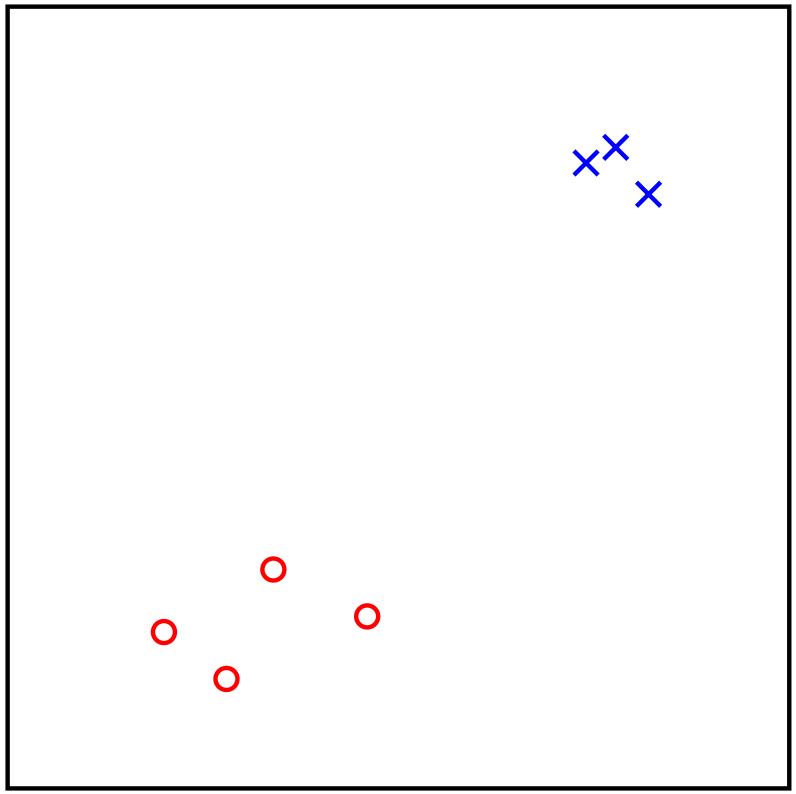
Examples

- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction...

Supervised learning

Examples

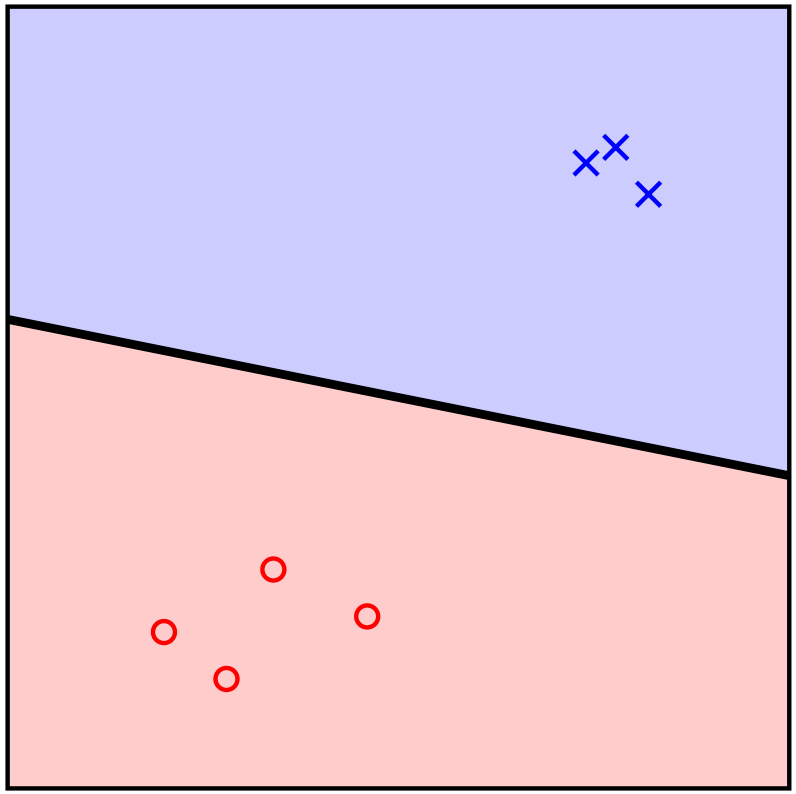
- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction...



Supervised learning

Examples

- **Classification** (**y** is categorical)
- **Regression** (**y** is scalar)
- Tons of applications:
  - Pattern recognition,
  - Prediction...



Plan

1 Supervised learning: Learning from labeled examples

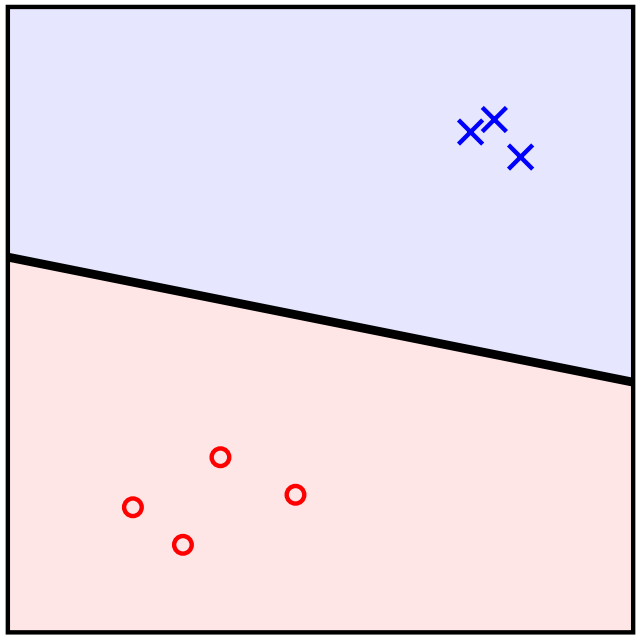
2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

Infinite solutions and priors

An ill-defined problem

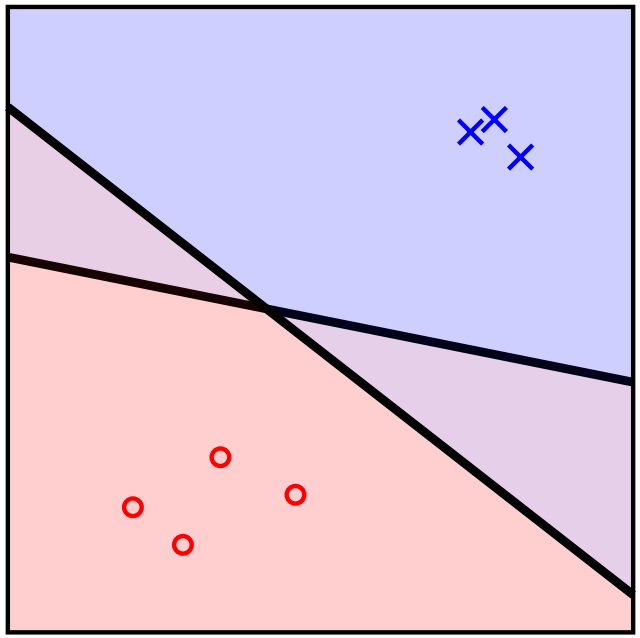
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors

An ill-defined problem

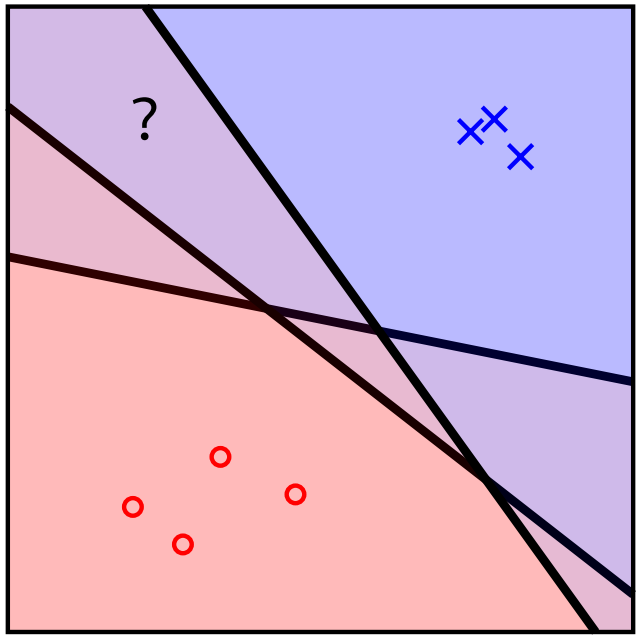
- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.



Infinite solutions and priors

An ill-defined problem

- An infinity of potential solutions, one must be the “best one” but is unreachable,
- $\Rightarrow$ requires **priors or constraints**.
- Training on more data is a way to reduce the number of potential solutions, but it is not always available.

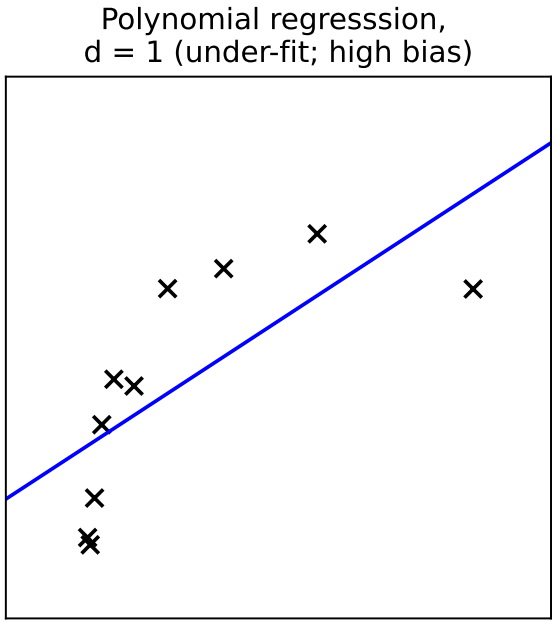


Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.

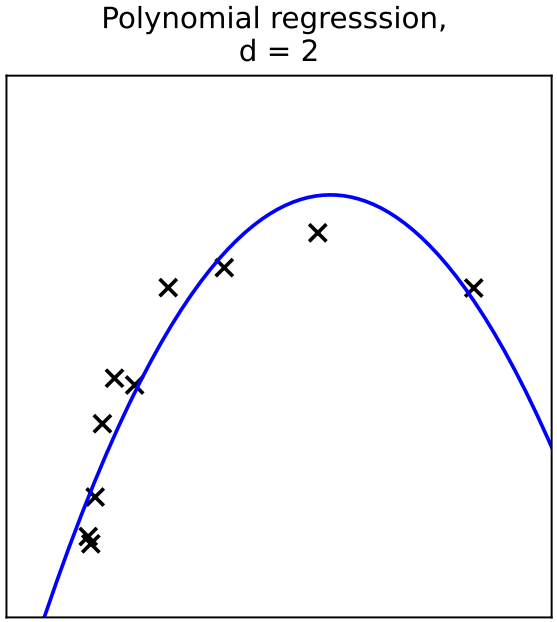


Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.

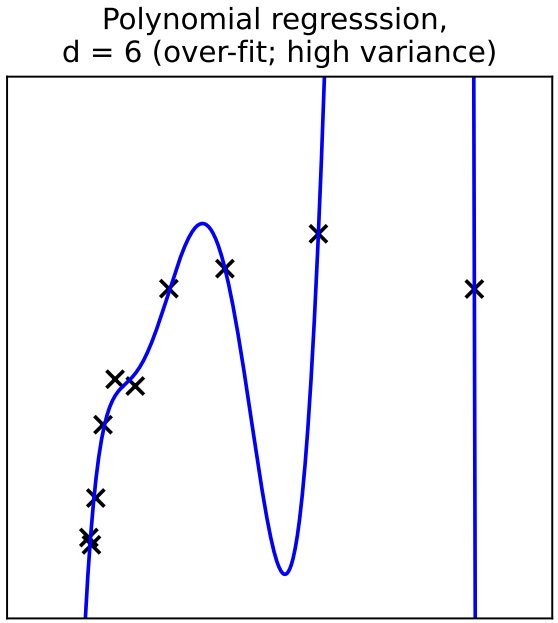


Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!

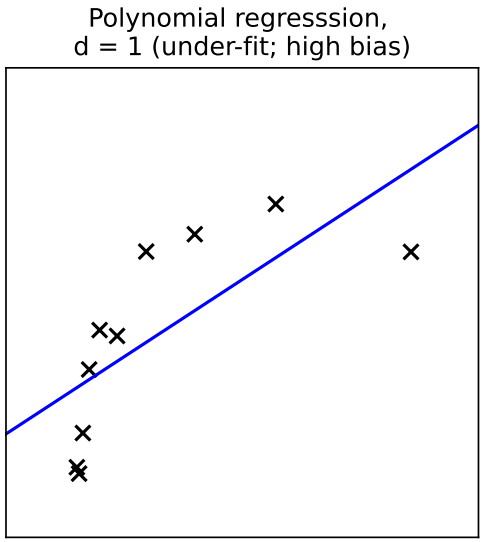
- Bias: Error from **erroneous assumptions** in the learning algorithm.
- Variance: Error from **sensitivity to small fluctuations** in the training set.

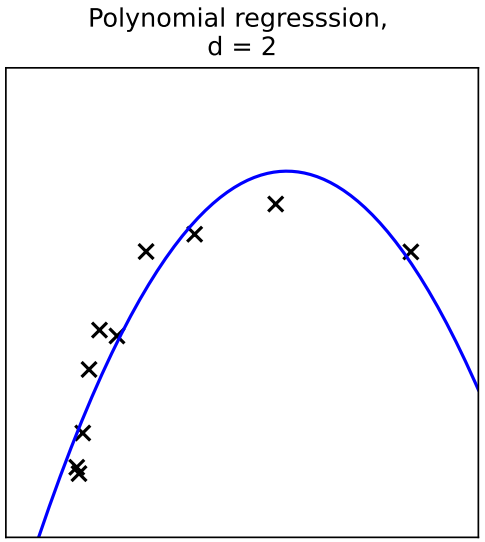


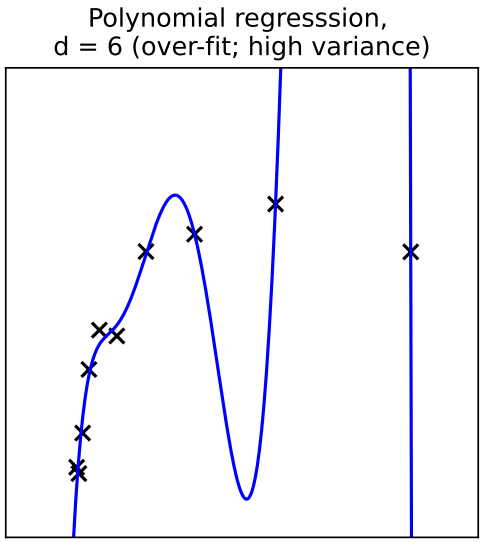
Bias & variance

Bias/variance trade-off

A **simple** solution that almost matches is better than a complex one that fully matches!



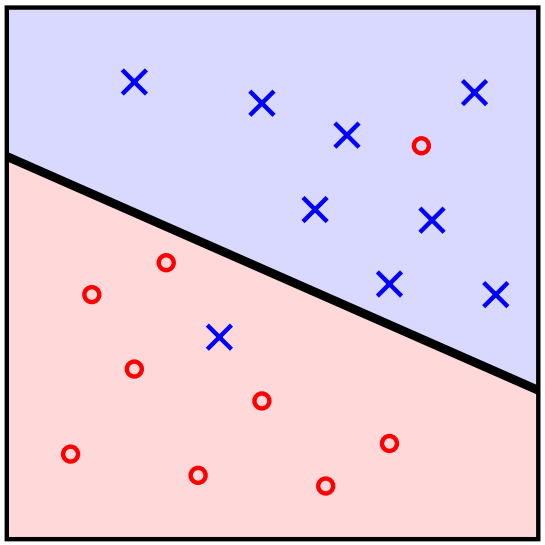




Overfitting

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.

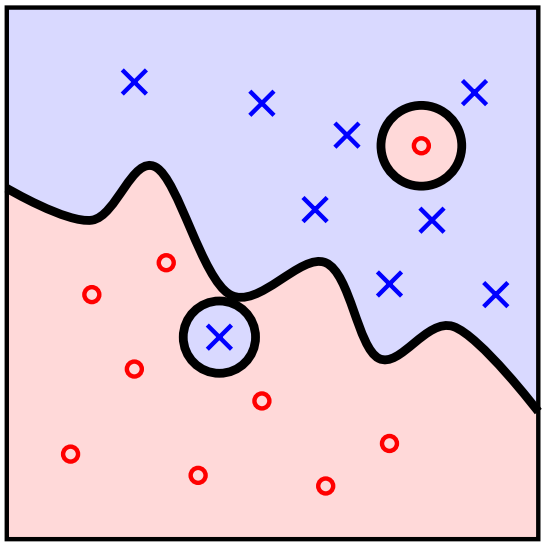


Simple model: **89%** of training points

Overfitting

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.

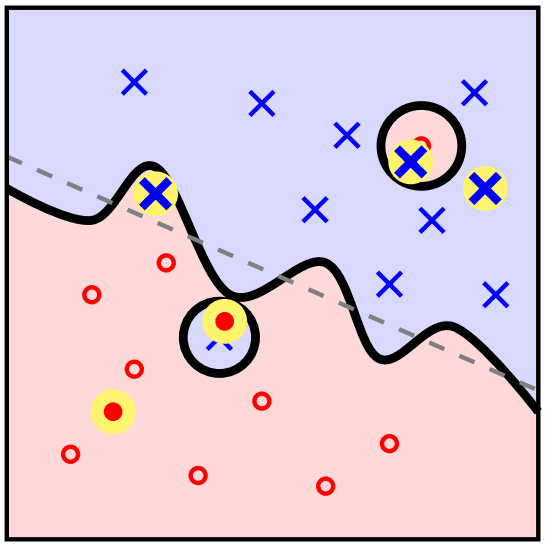


Complex model: **100%** of training points

Overfitting

Overfitting

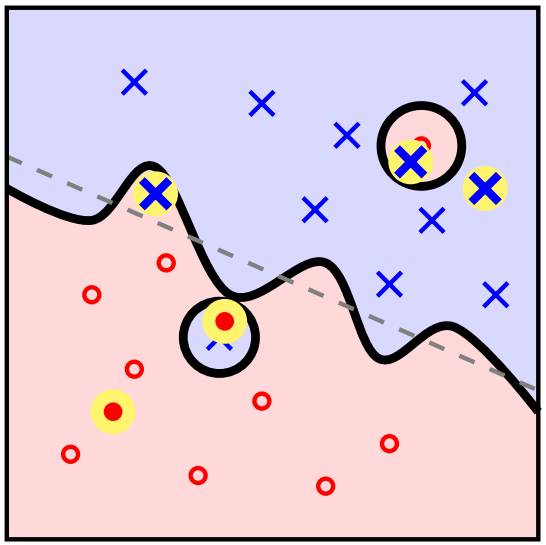
Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



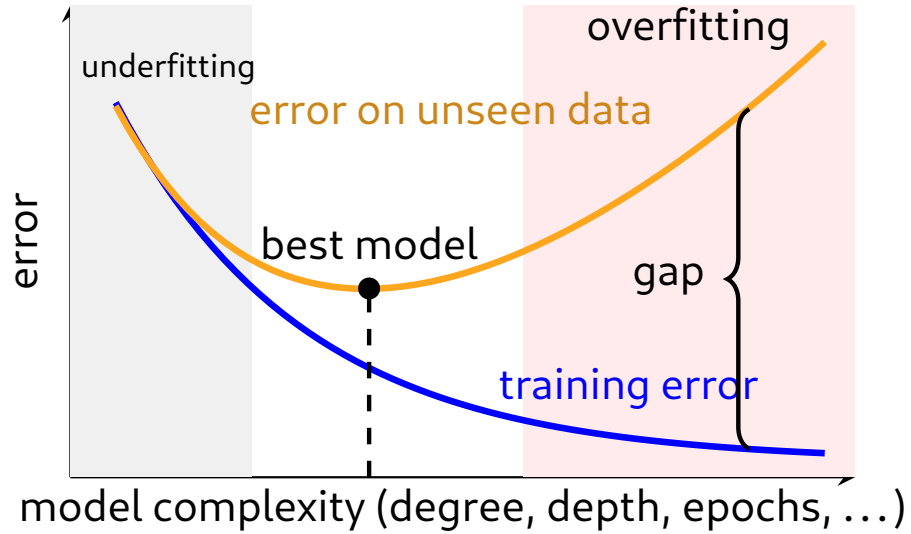
New points: simple **5/5**, complex **2/5**

Overfitting

Mimicking is not learning: a model can be **perfect on the training set** and **bad on new data**.



New points: simple **5/5**, complex **2/5**

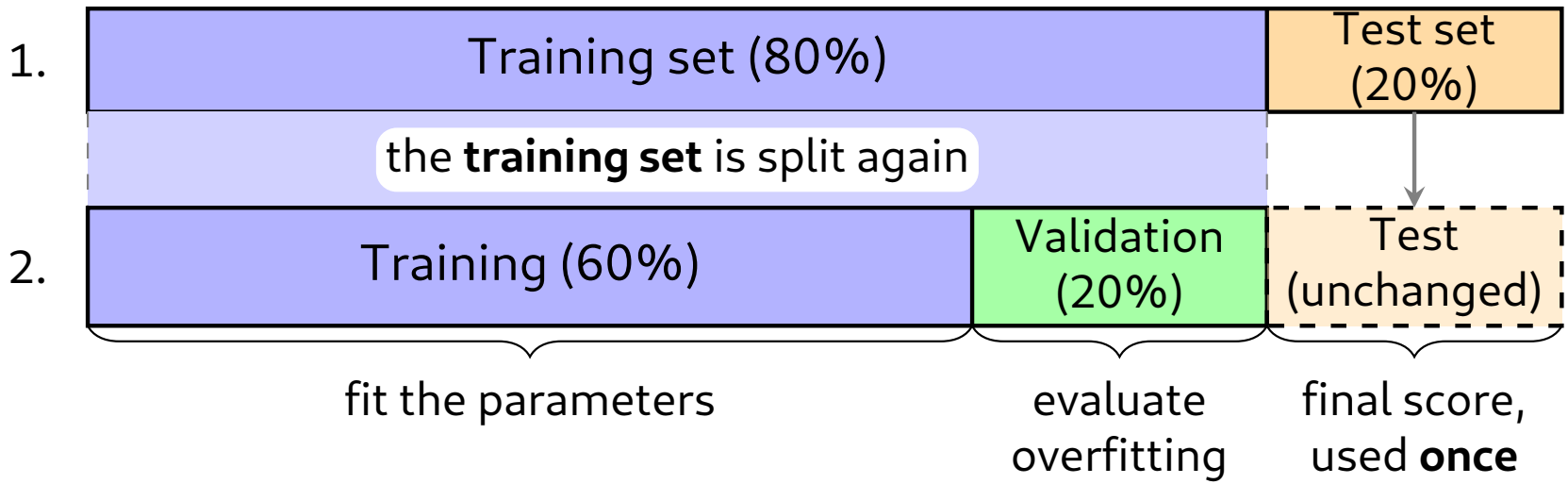


Symptom: the gap between training error and error on unseen data **grows**.

A validation set to detect overfitting

Validation set: a third subset of the data

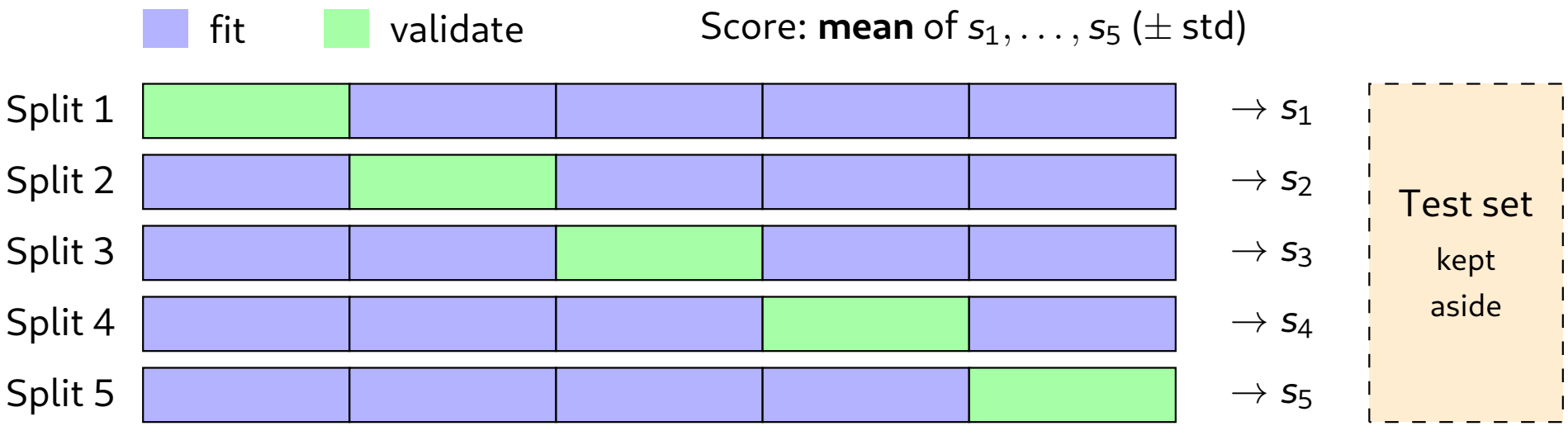
Split the **training** set again, into a part used to fit and a part used to **validate**.



Cross-validation

k-fold cross-validation (here $k = 5$)

With few data, a single validation split is noisy and wastes examples.
$\Rightarrow$ Cut the training set in k folds; each fold is used **once** for validation.



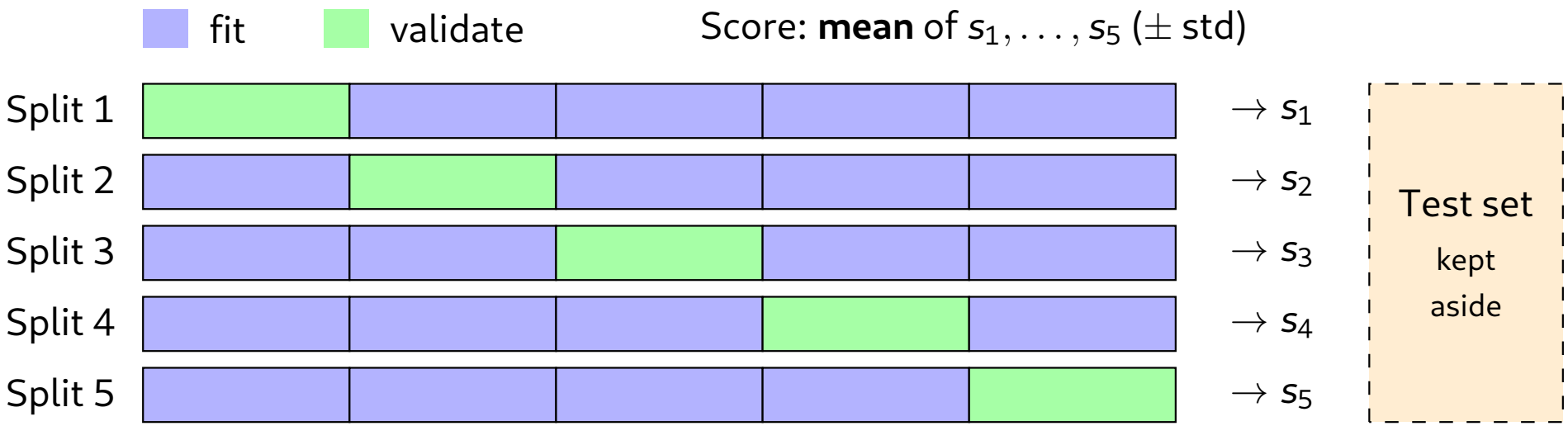
Cross-validation can also replace the train/test split

With few data, each fold can be used in turn as the **test** set.
Never train and test on the same folds.

Cross-validation

k-fold cross-validation (here $k = 5$)

With few data, a single validation split is noisy and wastes examples.
$\Rightarrow$ Cut the training set in k folds; each fold is used **once** for validation.



Cross-validation can also replace the train/test split

With few data, each fold can be used in turn as the **test** set.
Never train and test on the same folds.

High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

$d = 1$ | $d = 2$ | $d = 3$

ball: **100%** of the cube | **79%** | **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d \Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

see https://youtu.be/dZrGXYty3qc?t=533

High-dimensional data (1/2)

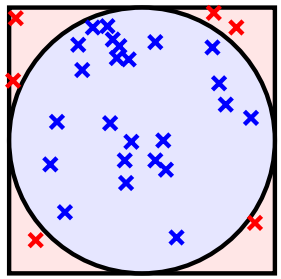
Curse of dimensionality

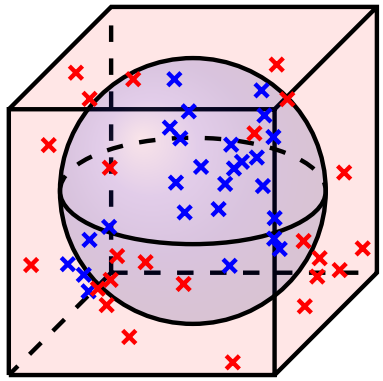
- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

$d = 1$ | $d = 2$ | $d = 3$







ball: **100%** of the cube | **79%** | **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d \Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

see https://youtu.be/dZrGXYty3qc?t=533

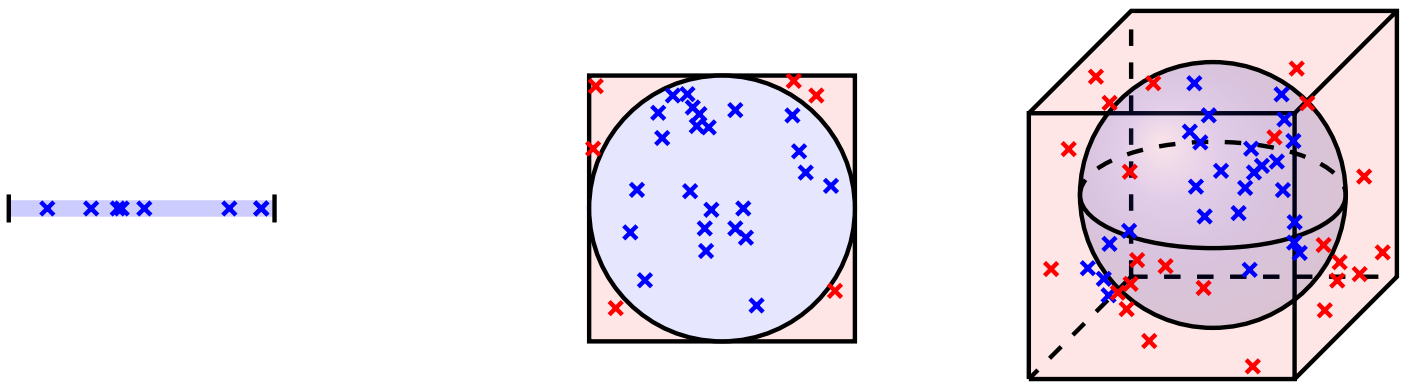
High-dimensional data (1/2)

Curse of dimensionality

- Geometry is not intuitive in **high dimension**,
- Efficient methods in 2D are not necessarily still valid.

Random points in a cube: inside or outside the ball it contains

$d = 1$ $d = 2$ $d = 3$



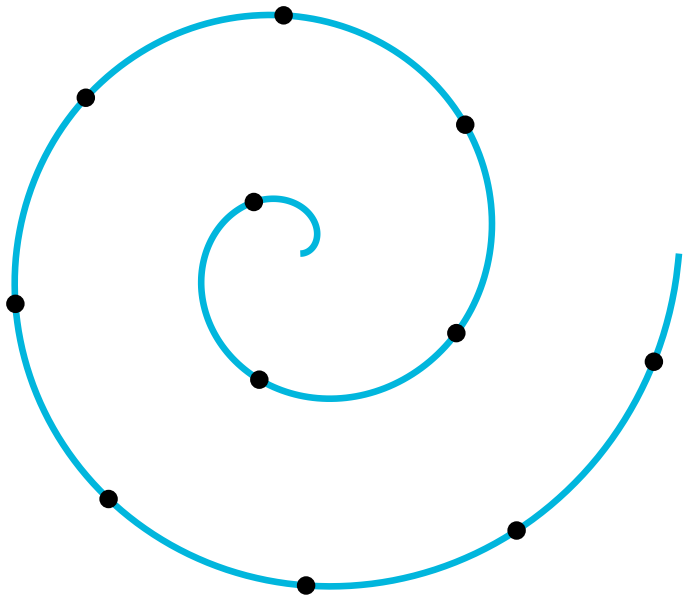
ball: **100%** of the cube **79%** **52%**

$V_{\text{ball}}/V_{\text{cube}} = \frac{\pi^{d/2}}{2^d\,\Gamma(d/2+1)}$: $d = 10$: **0.25%**, $d = 100$: $\approx$ **10^{-68}%**

$\Rightarrow$ **Almost all the volume is in the corners, far from the center!**

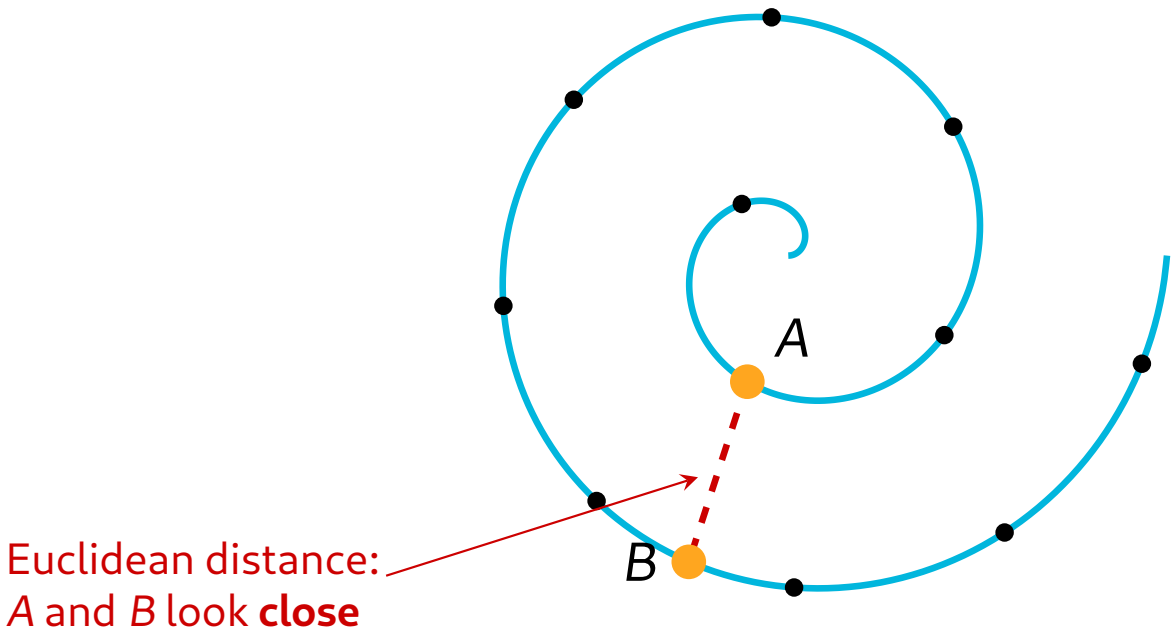
see `https://youtu.be/dZrGXYty3qc?t=533`

High-dimensional data (2/2): data lives on manifolds



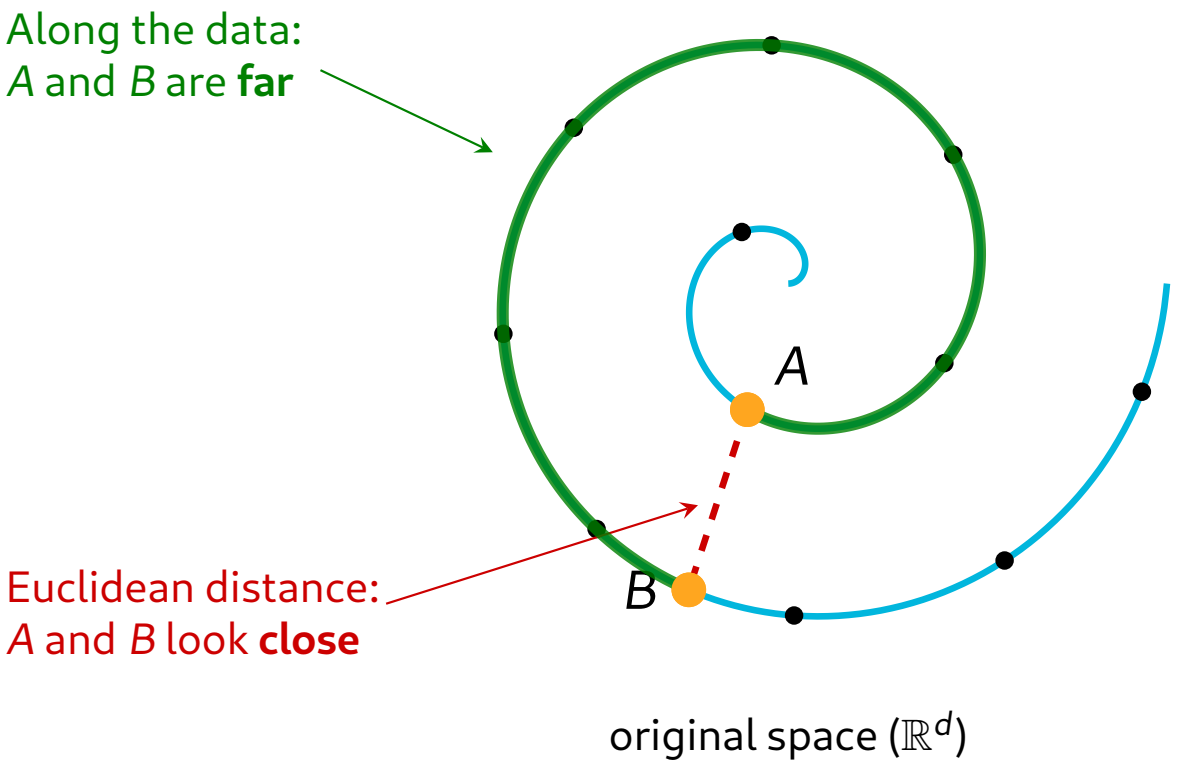
original space ($\mathbb{R}^d$)

High-dimensional data (2/2): data lives on manifolds

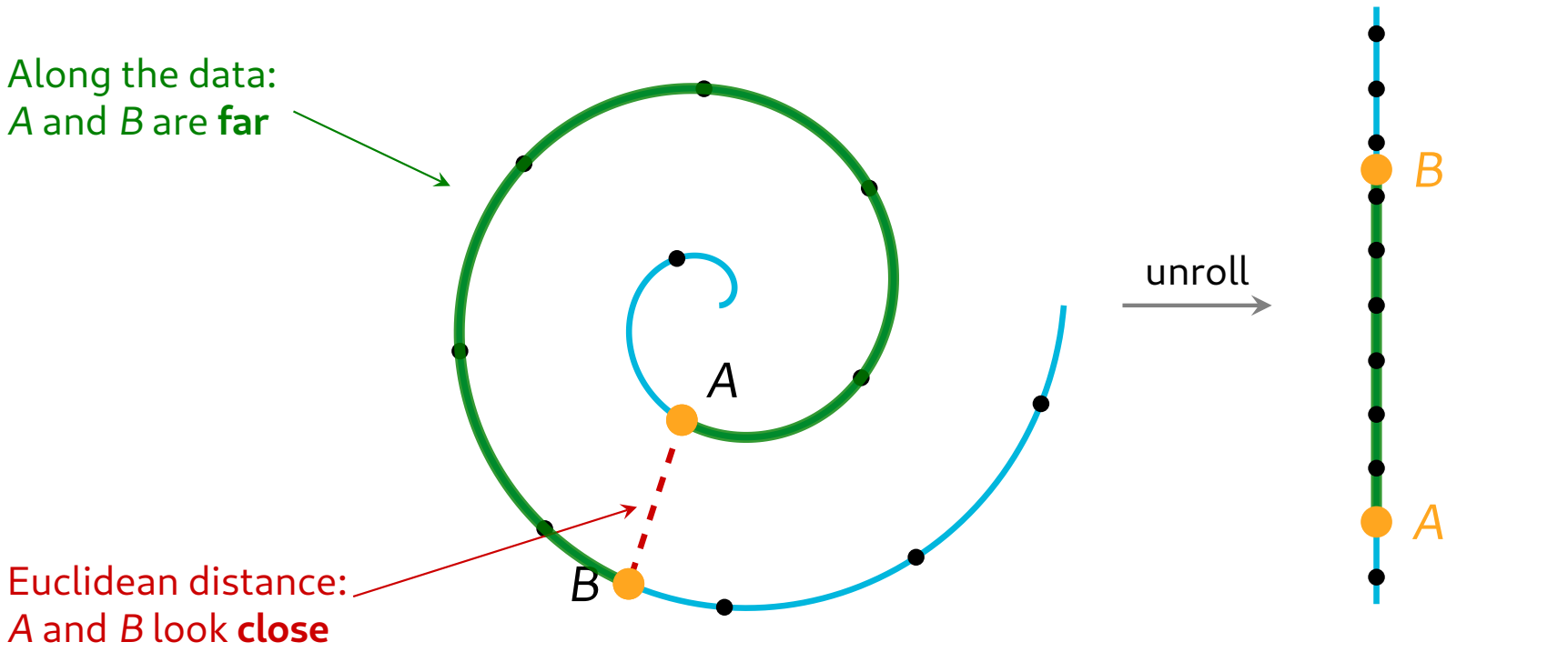


original space ($\mathbb{R}^d$)

High-dimensional data (2/2): data lives on manifolds



High-dimensional data (2/2): data lives on manifolds



The natural space of data may not always be suited to represent data!
$\Rightarrow$ This is why we use feature descriptions, or embeddings, to represent data!

Time and material resources

Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx$ 2h45 on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

Time and material resources

Computation time

Example on ImageNet, simply going through all images:

- $n = 10.000.000$, $d \approx 1.000.000$,
- $\approx 10^{13}$ elementary operations,
- $\approx$ 2h45 on a modern processor.

Scalability

- Finding the best solution to a problem would be feasible with unlimited computation time,
- But searching through the space of possible functions is often **untractable**,
- Solutions must be computationally reasonable, which is the true challenge today.

Plan

1 Supervised learning: Learning from labeled examples

2 Challenges of supervised learning

3 Evaluation: ground truth and metrics

Evaluation: ground truth vs. estimation

Ground truth $\mathbf{y}$

The **reference** answer: given by a human expert, a sensor, or a measurement protocol.
$\Rightarrow$ what we *assume* to be correct.

Estimation $\hat{\mathbf{y}} = f(\mathbf{x})$

The answer **produced by the model**.
$\Rightarrow$ available for any $\mathbf{x}$, even without label.

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

Why do we need metrics?

The problem

Comparing $\hat{\mathbf{y}}$ and $\mathbf{y}$ example by example does not scale: a test set has hundreds or thousands of examples.

$\Rightarrow$ we need to **summarise** the comparison into a few numbers.

What we use these numbers for

- **Choosing** between two algorithms (KNN or a decision tree?),
- **Tuning** hyperparameters (which value of k?),
- **Detecting** overfitting (compare train score vs. test score),
- **Deciding** whether the model is good enough to be used at all.

Loss $\neq$ metric

Loss $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

What the algorithm **minimises** during training. Must be optimisable, not necessarily interpretable.

Metric

What **you** report and interpret, on the test set. Must be meaningful for the application.

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$

A first metric: accuracy

Definition

The fraction of test examples that are correctly predicted:

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of examples}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{\mathbf{y}}_i = \mathbf{y}_i\right]$$

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat
	✓	✓	×	✓	✓

$$\text{accuracy} = 4/5 = \mathbf{80\%}$$

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy 10 sick

A very lazy model: "everybody is healthy"

- Accuracy $= 990/1000 =$ **99%**,
- Number of sick patients detected: **0**.

$\Rightarrow$ we need metrics that look at **each class separately**, and at the **type** of error being made.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy 10 sick

A very lazy model: "everybody is healthy"

- Accuracy $= 990/1000 =$ **99%**,
- Number of sick patients detected: **0**.

$\Rightarrow$ we need metrics that look at **each class separately**, and at the **type** of error being made.

Why accuracy is not enough

A screening test

1 000 patients, 10 of them are actually sick.



990 healthy 10 sick

A very lazy model: "everybody is healthy"

- Accuracy $= 990/1000 =$ **99%**,
- Number of sick patients detected: **0**.

$\Rightarrow$ we need metrics that look at **each class separately**, and at the **type** of error being made.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat

With "cat" as positive: $TP = 2$, $FP = 0$, $FN = 1$

$\text{precision} = 2/2 = \mathbf{100\%}$ but $\text{recall} = 2/3 \approx \mathbf{67\%}$

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \qquad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.

Precision and recall

Definition (for **one** class, chosen as "positive")

$$\text{precision} = \frac{TP}{TP + FP}$$

Among the examples **predicted** positive, how many really are?

$$\text{recall} = \frac{TP}{TP + FN}$$

Among the **truly** positive examples, how many are found?

TP: true positives, *FP*: false positives (*false alarms*), *FN*: false negatives (*missed*).

Test example	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$
Ground truth $\mathbf{y}$	cat	dog	cat	dog	cat
Estimation $\hat{\mathbf{y}}$	cat	dog	dog	dog	cat

With "cat" as positive: $TP = 2$, $FP = 0$, $FN = 1$

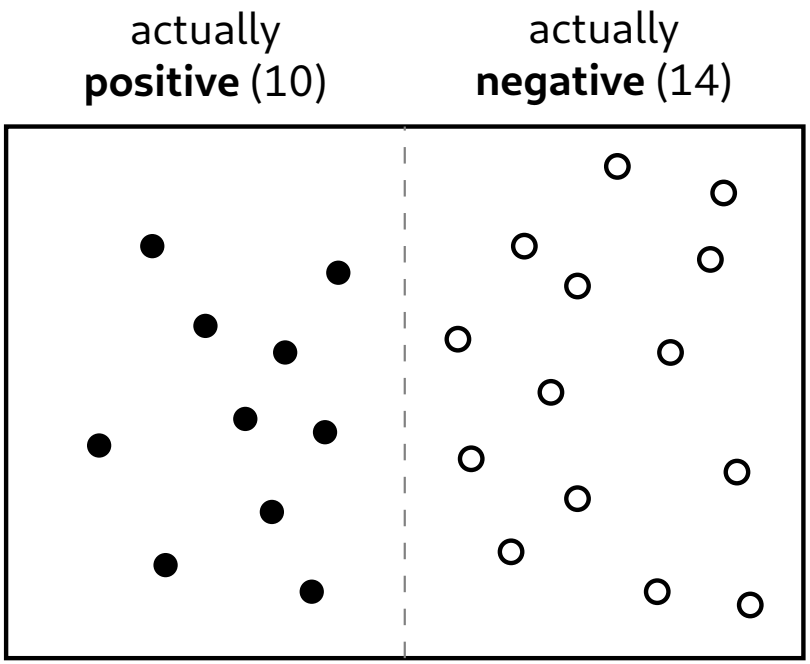
$\text{precision} = 2/2 = \mathbf{100\%}$ but $\text{recall} = 2/3 \approx \mathbf{67\%}$

F1-score: summarising both in one number

$$F_1 = 2 \cdot \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \qquad \text{here } F_1 = 2 \cdot \frac{1 \times 0.67}{1 + 0.67} \approx \mathbf{80\%}$$

Harmonic mean $\Rightarrow$ collapses as soon as **one** of the two is low.

Precision and recall: the intuition

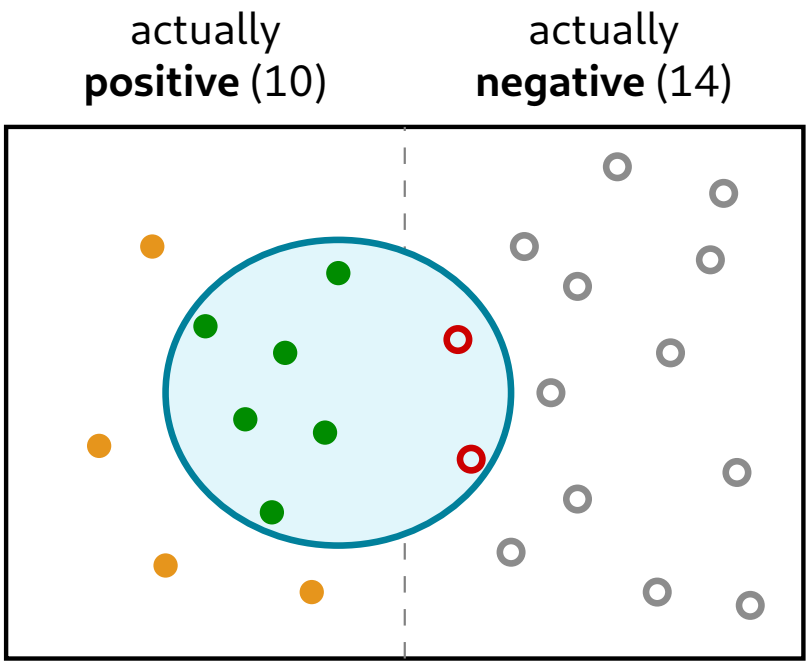


A test set of 24 examples:

- 10 positive examples (●),
- 14 negative examples (○).

The model must find the positive ones.

Precision and recall: the intuition



A first model

- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

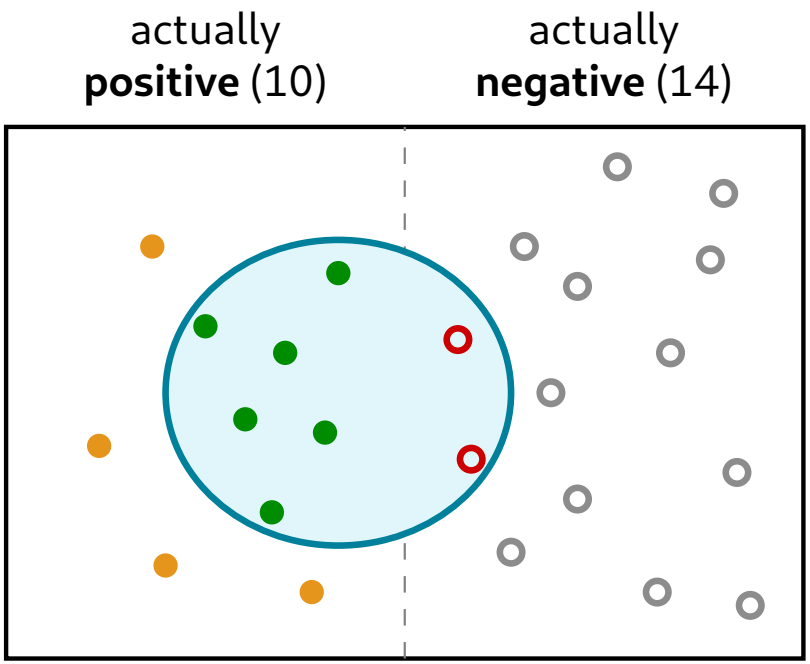
$$\text{precision} = \frac{TP}{TP+FP} = \frac{6}{8} = \mathbf{75\%}$$

share of the ellipse which is really positive

$$\text{recall} = \frac{TP}{TP+FN} = \frac{6}{10} = \mathbf{60\%}$$

share of the positives which is inside the ellipse

Precision and recall: the intuition



A first model

- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

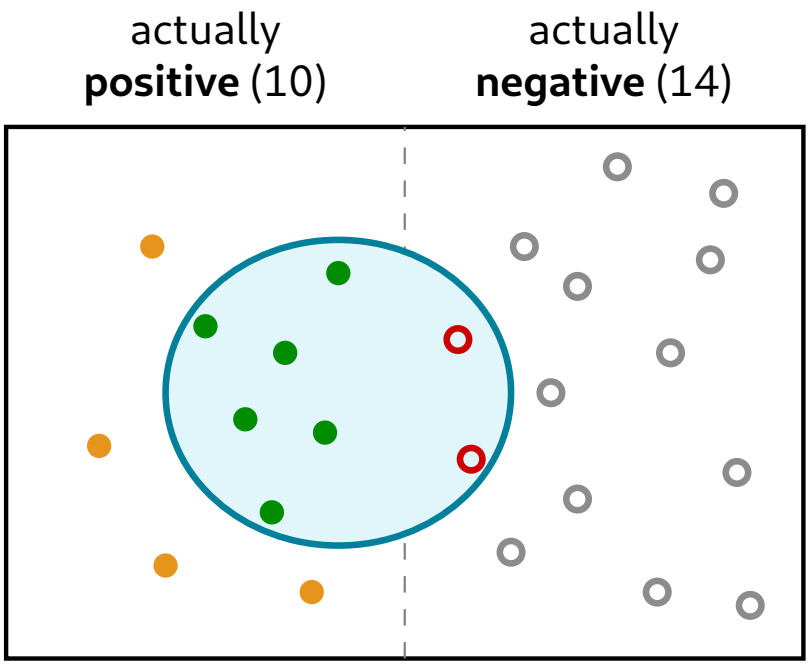
$$\text{precision} = \frac{TP}{TP+FP} = \frac{6}{8} = \mathbf{75\%}$$

share of the ellipse which is really positive

$$\text{recall} = \frac{TP}{TP+FN} = \frac{6}{10} = \mathbf{60\%}$$

share of the positives which is inside the ellipse

Precision and recall: the intuition



A first model

- $TP = 6$: positive, found
- $FN = 4$: positive, missed
- $FP = 2$: negative, false alarm
- $TN = 12$: negative, left out

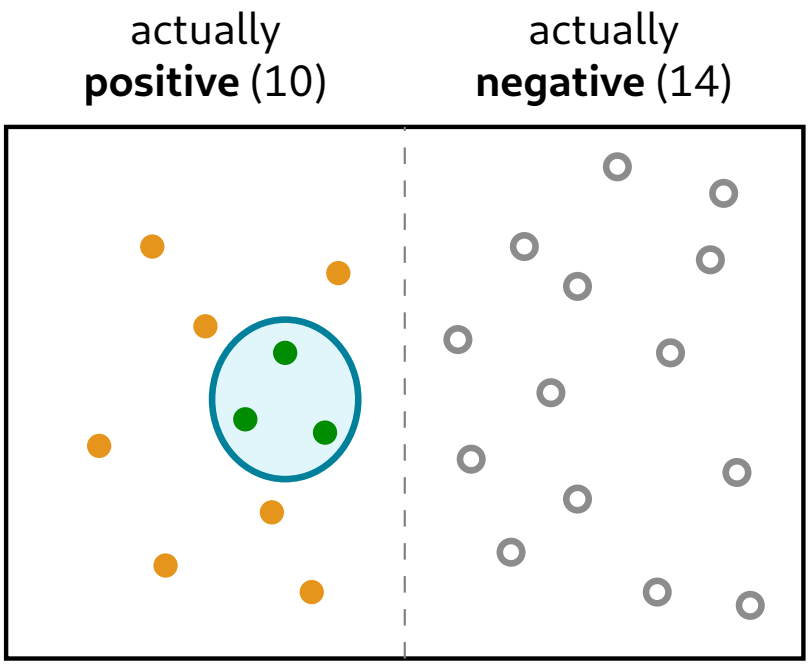
$$\text{precision} = \frac{TP}{TP+FP} = \frac{6}{8} = \mathbf{75\%}$$

share of the ellipse which is really positive

$$\text{recall} = \frac{TP}{TP+FN} = \frac{6}{10} = \mathbf{60\%}$$

share of the positives which is inside the ellipse

Precision and recall: the intuition



ellipse = **predicted** positive

A cautious model

- $TP = 3$: positive, found
- $FN = 7$: positive, missed
- $FP = 0$: negative, false alarm
- $TN = 14$: negative, left out

$$\text{precision} = \frac{TP}{TP+FP} = \frac{3}{3} = \mathbf{100\%}$$

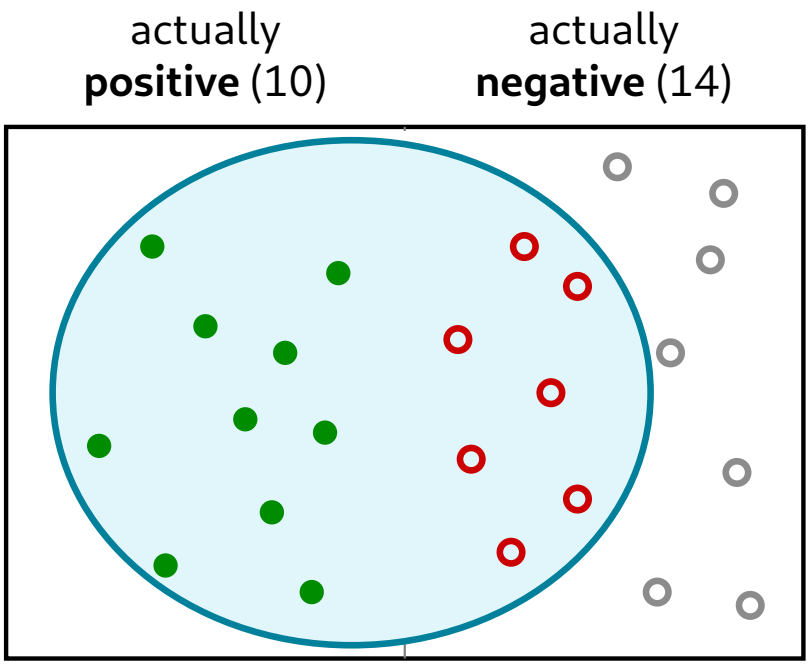
share of the ellipse which is really positive

$$\text{recall} = \frac{TP}{TP+FN} = \frac{3}{10} = \mathbf{30\%}$$

share of the positives which is inside the ellipse

Shrinking the ellipse: precision ↗, recall ↘. Inflating it: the opposite.

Precision and recall: the intuition



A permissive model

- $TP = 10$: positive, found
- $FN = 0$: positive, missed
- $FP = 7$: negative, false alarm
- $TN = 7$: negative, left out

$$\text{precision} = \frac{TP}{TP+FP} = \frac{10}{17} = \mathbf{59\%}$$

share of the ellipse which is really positive

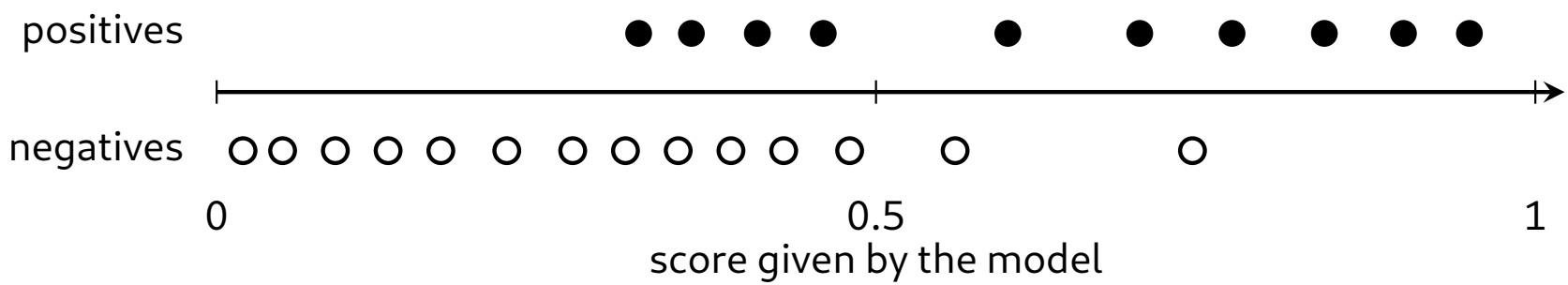
$$\text{recall} = \frac{TP}{TP+FN} = \frac{10}{10} = \mathbf{100\%}$$

share of the positives which is inside the ellipse

Shrinking the ellipse: precision ↗, recall ↘. Inflating it: the opposite.

Precision and recall: a trade-off

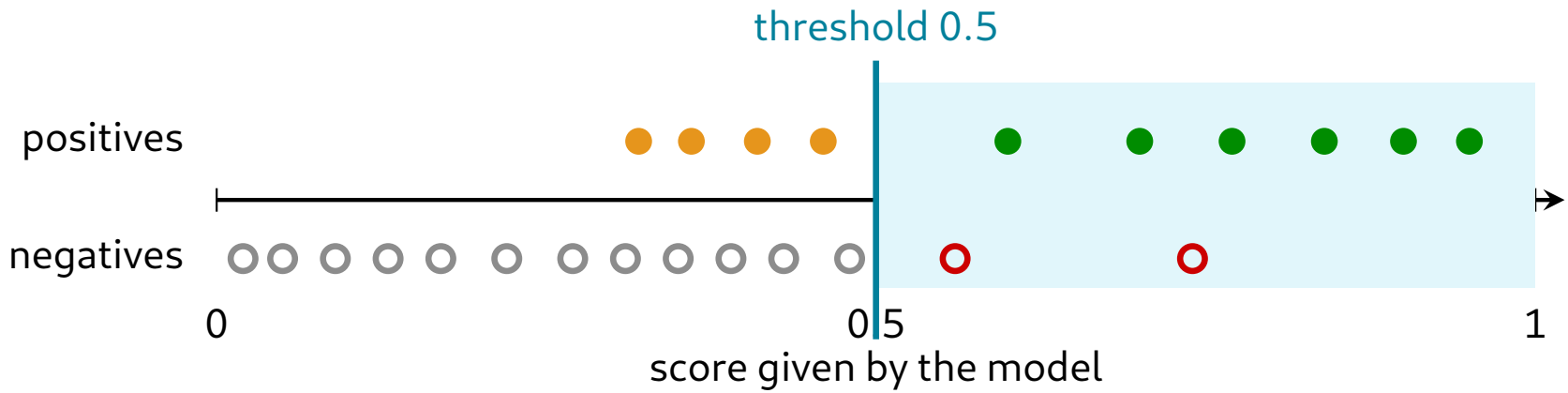
The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**

Precision and recall: a trade-off

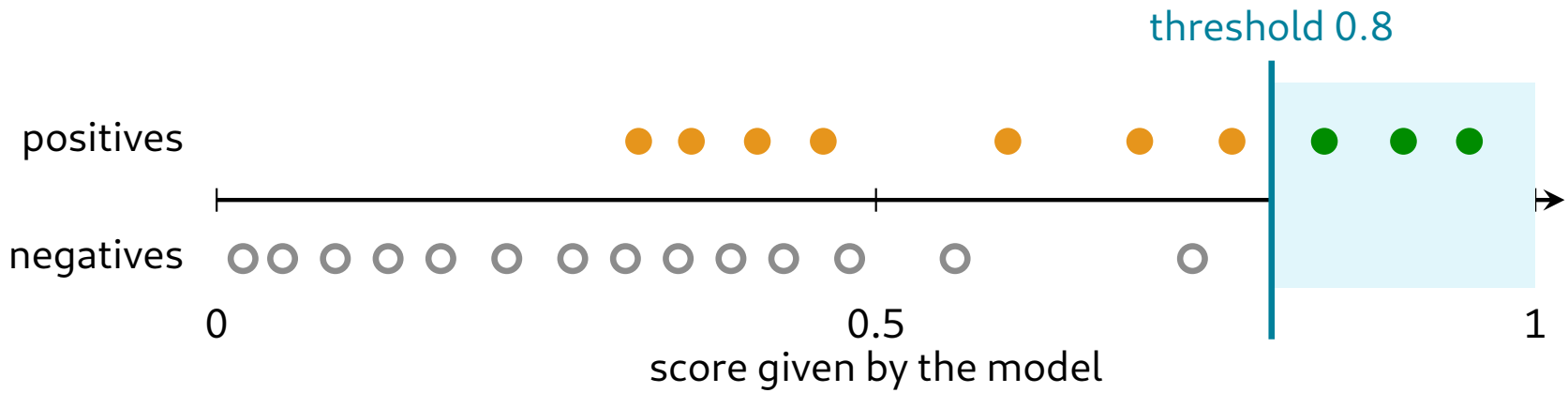
The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**

Precision and recall: a trade-off

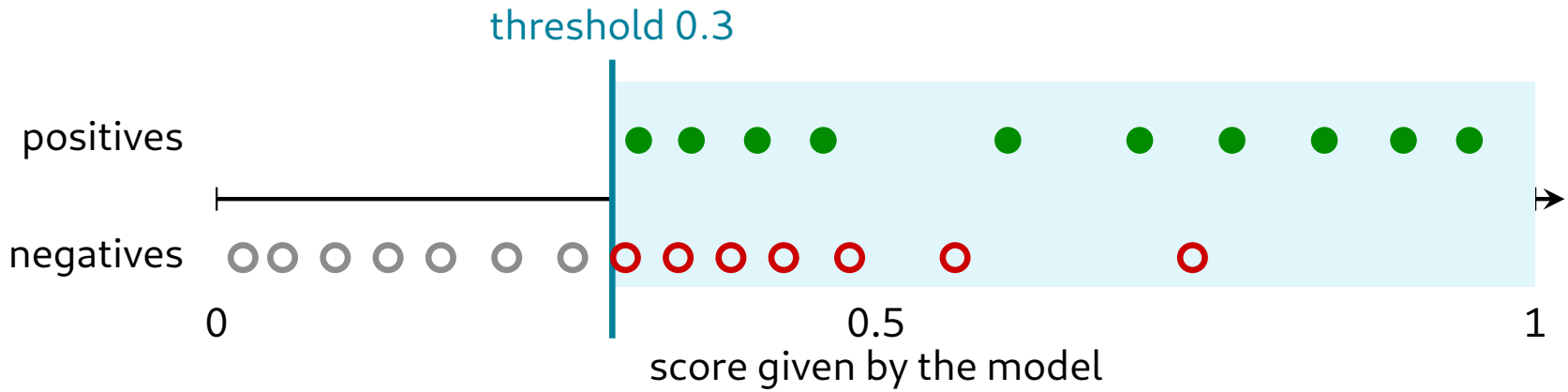
The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**

Precision and recall: a trade-off

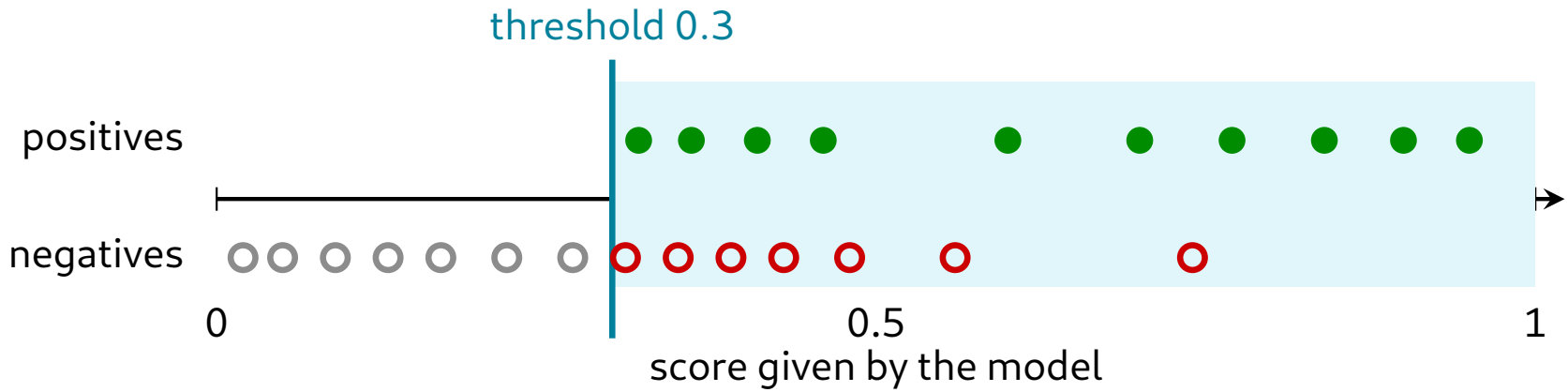
The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



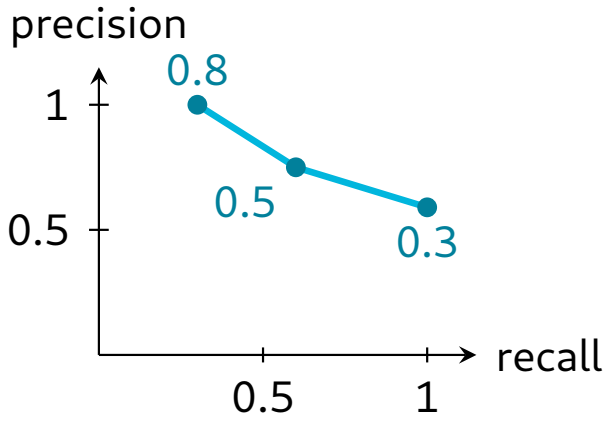
threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**

Precision and recall: a trade-off

The model outputs **one score** (e.g. the probability of being positive), and predicts "positive" when the score is above a **threshold**.



threshold	*TP*	*FP*	*FN*	precision	recall	F_1
0.5 (default)	6	2	4	75%	60%	67%
0.8 (cautious)	3	0	7	**100%**	30%	46%
0.3 (permissive)	10	7	0	59%	**100%**	**74%**



One model, many thresholds: this draws the **precision/recall curve**.

What about several classes?

One class at a time ("one-vs-rest")

For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx 83\%$, micro precision $= 80\%$.

What about several classes?

One class at a time ("one-vs-rest")

For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx 83\%$, micro precision $= 80\%$.

What about several classes?

One class at a time ("one-vs-rest")

For each class c: treat c as **positive** and **all the others** as negative.
$\Rightarrow$ one precision and one recall **per class**.

	precision	recall
class "cat"	$2/2 = 100\%$	$2/3 \approx 67\%$
class "dog"	$2/3 \approx 67\%$	$2/2 = 100\%$

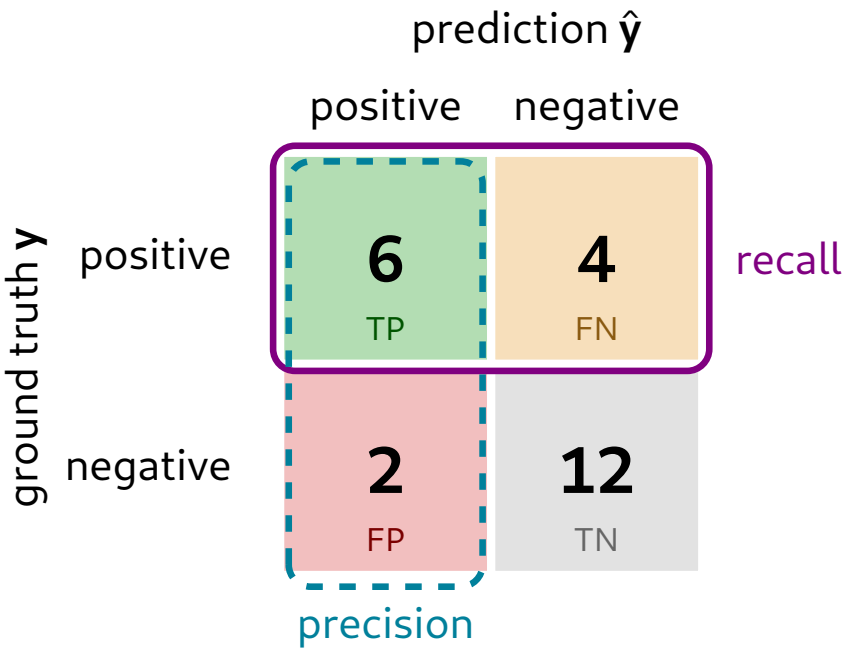
Averaging into a single number

- **macro**: plain mean over classes $\rightarrow$ every class counts the same,
- **weighted**: mean weighted by the number of examples per class,
- **micro**: pool the *TP*, *FP*, *FN* of all classes.

Here: macro precision $\approx 83\%$, micro precision $= 80\%$.

Confusion matrix

All the errors at once: **rows** = ground truth **y**, **columns** = prediction $\hat{\mathbf{y}}$ (convention of `sklearn`).



Two classes (the model of the previous slides):

$\text{recall} = \frac{6}{6+4} = 60\%$ (first row)

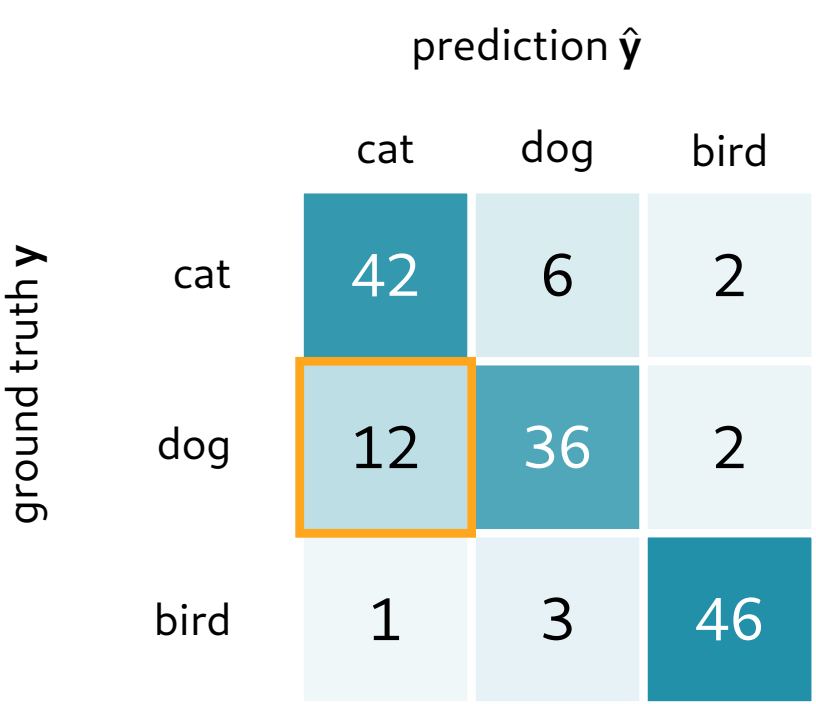
$\text{precision} = \frac{6}{6+2} = 75\%$ (first column)

$\text{accuracy} = \frac{6+12}{24} = 75\%$ (diagonal)

Confusion matrix

Confusion matrix

All the errors at once: **rows** = ground truth **y**, **columns** = prediction $\hat{\mathbf{y}}$ (convention of `sklearn`).



Several classes (50 test images per class):

- **diagonal**: correct predictions,
- **off diagonal**: *which* classes are confused,

  → **12 dogs** predicted as cats, but only 6 cats predicted as dogs.

accuracy $= \frac{42+36+46}{150} \approx 83\%$

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with `sklearn`

- Generate data with `make_blobs`, split it with `train_test_split`,
- The `sklearn` API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for *your* application.

Lab Session 2: Supervised Learning

Part 1 — Toy dataset, with `sklearn`

- Generate data with `make_blobs`, split it with `train_test_split`,
- The `sklearn` API: `fit` / `predict` / `score`,
- A first classifier (K-Nearest Neighbors): influence of k on train and test scores $\Rightarrow$ bias/variance in practice,
- Metrics: `classification_report` and `confusion_matrix`,
- Visualise the **decision boundaries**.

Then, do the same for a different dataset and a different classifier. Compare all results, and notice that they significantly differ!

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.
$\Rightarrow$ define your task as a classification (or regression) problem, check your class balance, and choose the metrics that make sense for *your* application.